```
}

object LogProcessingGuardian {

  def apply(directories: Vector[String]) =
    Behaviors
      .setup[Nothing] { context =>                                    ❷
        directories.foreach { directory =>

          val fileWatcher: ActorRef[FileWatcher.Command] =
            context.spawnAnonymous(FileWatcher(directory))
          context.watch(fileWatcher)                                  ❸
        }
        Behaviors
          .receiveMessage[Nothing] {
            case _: Any =>
              Behaviors.ignore
          }
          .receiveSignal {
            case (context, Terminated(actorRef)) =>
              ...                                                     ❹
              Behaviors.same
          }
      }
}
```

❶ Input. These directories could come from args using Main instead of extending from App.

❷ Not meant to receive messages. There's a Nothing protocol.

❸ Monitors each file watcher

❹ Checks that not all file watchers are terminated. If no file watchers are left, shuts down the system.

NOTE In this example, you create an actor with `spawnAnonymous`. This method is the same as `spawn`, but you don't need to give the actor a name since a random name is created for you. For modeling, this is a good option because it is simpler. However, in a project that will be deployed and used, it is recommended to use `spawn` and give the actor a name so you can make sense of it when debugging the application.